Data Structures

BS (CS) _Fall_2025

Lab_14 Task

Learning Objectives:

1. Heaps and Graphs (Adjacency Matrix)

Lab Task 1

Problem Statement

You are required to design and implement a templated Heap class in C++ that performs the following operations related to Max Heaps. The class should internally manage a dynamically allocated array to store heap elements. The array should automatically resize (double the size when full, halve when sparse).

Heaps

1. **Convert an Array into a Binary Tree**
    - Write a function to take an input array and represent it as a Binary Tree structure.
2. **Implement the Heapify() Function**
    - Implement a function heapify() that converts a binary tree (represented as an array) into a valid Max Heap.
    - The function should ensure that the heap property (parent >= children) is maintained at all times.
3. **Implement the buildMaxHeap() Function**
    - Write a function void buildMaxHeap() that takes an unsorted array and builds a Max Heap using repeated calls to heapify().

Implement the following Heap Operations

Implement each of the following functions:

- extractMin() → Removes and returns the minimum element from the heap.
- extractMax() → Removes and returns the maximum element from the heap.
- peekMin() → Returns the minimum element without removing it.
- peekMax() → Returns the maximum element without removing it.
- insertKey() → Inserts a new key into the heap while maintaining the Max Heap property.
- deleteKey() → Deletes the element at index i from the heap.

Google Test Cases:

```
TEST(HeapTest, BuildMaxHeapBasic) {
    int arr[] = {10, 3, 5, 2, 8, 15, 7};
    Heap<int> heap;
    heap.buildMaxHeap(arr, 7);
    EXPECT_TRUE(heap.isMaxHeap());
```

```
    EXPECT_EQ(heap.peekMax(), 15);
}

TEST(HeapTest, AlreadySortedArray) {
    int arr[] = {5, 4, 3, 2, 1};
    Heap<int> heap;
    heap.buildMaxHeap(arr, 5);
    EXPECT_TRUE(heap.isMaxHeap());
    EXPECT_EQ(heap.peekMax(), 5);
}

TEST(HeapTest, ReverseSortedArray) {
    int arr[] = {1, 2, 3, 4, 5, 6, 7};
    Heap<int> heap;
    heap.buildMaxHeap(arr, 7);
    EXPECT_TRUE(heap.isMaxHeap());
    EXPECT_EQ(heap.peekMax(), 7);
}

TEST(HeapTest, ExtractMaxOperation) {
    int arr[] = {10, 3, 5, 2, 8, 15, 7};
    Heap<int> heap;
    heap.buildMaxHeap(arr, 7);
    int maxVal = heap.extractMax();
    EXPECT_EQ(maxVal, 15);
    EXPECT_TRUE(heap.isMaxHeap());
    EXPECT_EQ(heap.peekMax(), 10);
}

TEST(HeapTest, ExtractMinOperation) {
    int arr[] = {10, 3, 5, 2, 8, 15, 7};
    Heap<int> heap;
    heap.buildMaxHeap(arr, 7);
    int minVal = heap.extractMin();
    EXPECT_EQ(minVal, 2);
    EXPECT_TRUE(heap.isMaxHeap());
}

TEST(HeapTest, PeekOperations) {
    int arr[] = {10, 3, 5, 2, 8, 15, 7};
    Heap<int> heap;
    heap.buildMaxHeap(arr, 7);
    EXPECT_EQ(heap.peekMax(), 15);
    EXPECT_EQ(heap.peekMin(), 2);
}
```

```
TEST(HeapTest, InsertKeyDynamicResize) {
    Heap<int> heap(3);
    int arr[] = {5, 7, 9};
    heap.buildMaxHeap(arr, 3);
    heap.insertKey(10);
    EXPECT_TRUE(heap.isMaxHeap());
    EXPECT_EQ(heap.peekMax(), 10);
    heap.insertKey(12);
    EXPECT_TRUE(heap.isMaxHeap());
    EXPECT_EQ(heap.peekMax(), 12);
    EXPECT_GE(heap.getSize(), 5);
}

TEST(HeapTest, DeleteKeyByIndex) {
    int arr[] = {10, 3, 5, 2, 8, 15, 7};
    Heap<int> heap;
    heap.buildMaxHeap(arr, 7);
    heap.deleteKey(0);
    EXPECT_TRUE(heap.isMaxHeap());
    EXPECT_EQ(heap.getSize(), 6);
}

TEST(HeapTest, DuplicateElements) {
    int arr[] = {5, 5, 5, 5, 5};
    Heap<int> heap;
    heap.buildMaxHeap(arr, 5);
    EXPECT_TRUE(heap.isMaxHeap());
    EXPECT_EQ(heap.peekMax(), 5);
    EXPECT_EQ(heap.extractMax(), 5);
    EXPECT_TRUE(heap.isMaxHeap());
}

TEST(HeapTest, SingleElementHeap) {
    int arr[] = {42};
    Heap<int> heap;
    heap.buildMaxHeap(arr, 1);
    EXPECT_TRUE(heap.isMaxHeap());
    EXPECT_EQ(heap.peekMax(), 42);
    EXPECT_EQ(heap.extractMax(), 42);
    EXPECT_EQ(heap.getSize(), 0);
}

TEST(HeapTest, NegativeNumbers) {
    int arr[] = {-5, -10, 0, -3, -8};
```

```
    Heap<int> heap;
    heap.buildMaxHeap(arr, 5);
    EXPECT_TRUE(heap.isMaxHeap());
    EXPECT_EQ(heap.peekMax(), 0);
    EXPECT_EQ(heap.extractMax(), 0);
    EXPECT_TRUE(heap.isMaxHeap());
}

TEST(HeapTest, LargeHeapPerformance) {
    Heap<int> heap;
    const int N = 1000;
    int* arr = new int[N];
    for (int i = 0; i < N; i++) arr[i] = i;
    heap.buildMaxHeap(arr, N);
    delete[] arr;

    EXPECT_TRUE(heap.isMaxHeap());
    for (int i = 0; i < 100; i++) {
        int maxVal = heap.extractMax();
        EXPECT_GE(maxVal, 0);
    }
    EXPECT_TRUE(heap.isMaxHeap());
}

TEST(HeapTest, MixedInsertDeleteSequence) {
    int arr[] = {10, 20, 15, 30, 40};
    Heap<int> heap;
    heap.buildMaxHeap(arr, 5);
    EXPECT_TRUE(heap.isMaxHeap());

    heap.insertKey(50);
    heap.insertKey(5);
    EXPECT_TRUE(heap.isMaxHeap());
    EXPECT_EQ(heap.peekMax(), 50);

    heap.deleteKey(0); // delete largest
    EXPECT_TRUE(heap.isMaxHeap());

    int maxVal = heap.extractMax();
    EXPECT_TRUE(heap.isMaxHeap());
    EXPECT_LE(maxVal, 50);
}
```

Lab Task 2

You are working as a Software Engineer at a company developing **network analysis tools**. Your task is to implement a **graph representation using an Adjacency Matrix**. The graph should efficiently support basic operations, degree calculations, and subgraph analysis.

Requirements:

1. **addEdge()**
   Adds an edge between node u and node v. Should check if both nodes exist.
2. **deleteEdge()**
   Removes an edge if it exists.
3. **edgeExists()**
   Returns true if an edge exists, otherwise false.
4. **nodeExists()**
   Returns true if the node exists in the graph.
5. **printGraph()**
   Prints the **adjacency matrix** with proper formatting.
6. **degree()**
   Returns the **total degree** of the node (sum of connections).
7. **inDegree() / outDegree()**

   Returns the in-degree and out-degree of the node.
8. **isSubgraph(const GraphMatrix<T>& sub)**
   Returns true if all edges of sub exist in the main graph.

Constraints

- Assume you are working with **directed graphs.**
- Input validation: Do not allow adding or deleting edges between non-existent nodes.
- Use **templates** so that nodes can be integers, characters, or strings.

Google Test Cases:

```
#include <string>
#include <vector>
using namespace std;

TEST(GraphMatrixTest, AddAndEdgeExists_Int) {
    GraphMatrix<int> g(5);
```

```
    g.addEdge(0, 1);
    g.addEdge(1, 2);

    EXPECT_TRUE(g.edgeExists(0, 1));
    EXPECT_FALSE(g.edgeExists(1, 0));  // directed graph check
    EXPECT_FALSE(g.edgeExists(0, 3));
}

TEST(GraphMatrixTest, NodeExists_Int) {
    GraphMatrix<int> g(4);
    EXPECT_TRUE(g.nodeExists(0));
    EXPECT_TRUE(g.nodeExists(3));
    EXPECT_FALSE(g.nodeExists(5));
}

TEST(GraphMatrixTest, DeleteEdge_Int) {
    GraphMatrix<int> g(4);
    g.addEdge(0, 1);
    g.addEdge(2, 3);
    EXPECT_TRUE(g.edgeExists(0, 1));

    g.deleteEdge(0, 1);
    EXPECT_FALSE(g.edgeExists(0, 1));
    EXPECT_TRUE(g.edgeExists(2, 3));
}

TEST(GraphMatrixTest, PrintGraph_Int) {
    GraphMatrix<int> g(3);
    g.addEdge(0, 1);
    g.addEdge(1, 2);
    g.addEdge(0, 1); // duplicate
    g.printGraph();
}

TEST(GraphMatrixTest, StringTypeGraph) {
    GraphMatrix<string> g(3);
    g.addEdge(0, 2);
    EXPECT_TRUE(g.edgeExists(0, 2));
    EXPECT_FALSE(g.edgeExists(2, 0)); // directed check
    EXPECT_FALSE(g.edgeExists(1, 2));
}

TEST(GraphMatrixTest, DegreeCalculations) {
    GraphMatrix<int> g(4);
    g.addEdge(0, 1);
```

```
    g.addEdge(0, 2);
    g.addEdge(1, 2);
    g.addEdge(2, 0);

    EXPECT_EQ(g.degree(0), 2 + 1);      // out + in
    EXPECT_EQ(g.outDegree(0), 2);
    EXPECT_EQ(g.inDegree(0), 1);

    EXPECT_EQ(g.degree(2), 1 + 2);      // out + in
    EXPECT_EQ(g.outDegree(2), 1);
    EXPECT_EQ(g.inDegree(2), 2);
}

TEST(GraphMatrixTest, IsSubgraph_Int) {
    GraphMatrix<int> g(4);
    g.addEdge(0, 1);
    g.addEdge(1, 2);
    g.addEdge(2, 3);

    GraphMatrix<int> sub(3);
    sub.addEdge(0, 1);
    sub.addEdge(1, 2);

    GraphMatrix<int> notSub(3);
    notSub.addEdge(0, 2);

    EXPECT_TRUE(g.isSubgraph(sub));
    EXPECT_FALSE(g.isSubgraph(notSub));
}

TEST(GraphMatrixTest, LargeGraphPerformance) {
    GraphMatrix<int> g(100);
    for (int i = 0; i < 99; i++) g.addEdge(i, i + 1);
    for (int i = 0; i < 99; i++) EXPECT_TRUE(g.edgeExists(i, i + 1));
    EXPECT_FALSE(g.edgeExists(0, 99));
}
```